

Modul 03: Entwurfsmuster kombiniert anwenden

Systemvoraussetzungen: Visual Studio, .NET

Ziel: Das Ziel dieses Labs ist es, die Prinzipien der Entwurfsmuster zu verinnerlichen und kombiniert anwenden zu können. Es soll eine Anwendung erstellt werden, die verschiedene Zahlungsmethoden verwendet, um eine Bestellung bezahlen zu können. Der Produktkorb soll schrittweise mit Produkten aufgefüllt werden können.

Hinweise: Bei diesen Labs geht es nicht um die vollständige Implementierung, sondern ein Verpublic void ständnis der Konzepte zu entwickeln. Bitte begründen Sie Ihre Entscheidungen und gewählte Designoptionen. Die Entwürfe können in der Feedbackrunde diskutiert werden.

1 Strategy-Pattern

Definiere eine Schnittstelle `IPaymentStrategy`, die eine Methode `Pay` enthält.

```
public interface IPaymentStrategy
{
    void Pay(decimal amount);
}
```

Erstelle dann die Klassen, welche die `IPaymentStrategy`-Schnittstelle implementieren und verschiedene Zahlungsmethoden wie Kreditkarte, PayPal und Banküberweisung enthalten.

2 Kontext als Fabrik erstellen

Erstelle eine Klasse `ShoppingCart`, die eine `IPaymentStrategy`-Instanz verwendet, um eine Zahlung durchzuführen. Hierbei soll sich die Klasse wie eine Factory verhalten, welche die entsprechende Zahlungsmethode selbst erstellen kann.

```
public void SetPaymentMethod(PaymentMethod paymentMethod)
    => _paymentStrategy = CreatePaymentStrategy(paymentMethod);

private IPaymentStrategy CreatePaymentStrategy(PaymentMethod paymentMethod)
    => paymentMethod switch
    {
        PaymentMethod.PayPal => new PayPalPayment(),
        PaymentMethod.CreditCard => new CreditCardPayment(),
        PaymentMethod.BankTransfer => new BankTransferPayment(),
        PaymentMethod.SelfPickup => new SelfPickupPaymentAdapter(),
        _ => throw new ArgumentException("Invalid payment method")
    };
};
```

3 Builder-Pattern erstellen

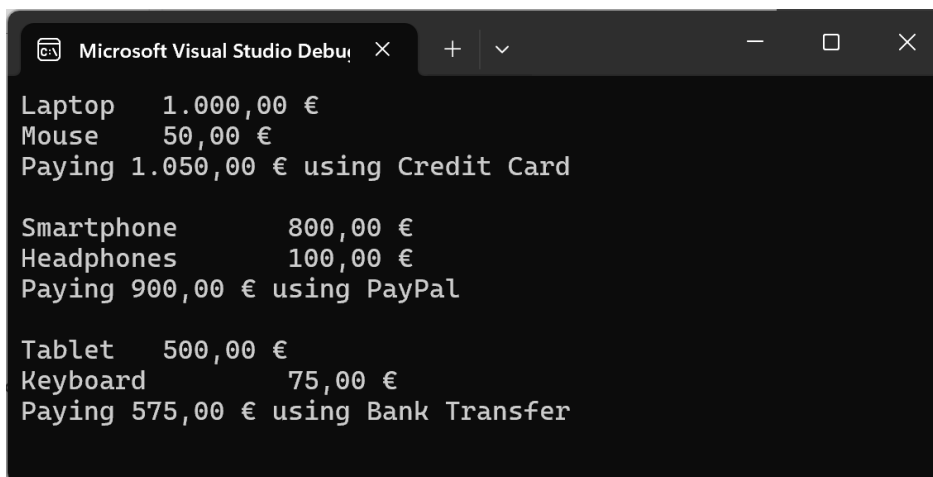
Erstelle eine ShoppingCartBuilder-Klasse, um den ShoppingCart schrittweise mit Produkten zu füllen und die Zahlungsmethode zu setzen.

```
var cart = new ShoppingCartBuilder()
    .AddProduct("Laptop", 1000.0)
    .AddProduct("Mouse", 50.0)
    .SetPaymentMethod(PaymentMethod.CreditCard)
    .Build();

cart.PrintInfo();
cart.Pay();
```

4 Implementierung testen

Eine einfache Konsolenanwendung soll die implementierte Logik testen sowie die verschiedenen Zahlungsmethoden und Ergebnisse ausgeben.



```
Microsoft Visual Studio Debug Console
Laptop    1.000,00 €
Mouse     50,00 €
Paying 1.050,00 € using Credit Card

Smartphone 800,00 €
Headphones 100,00 €
Paying 900,00 € using PayPal

Tablet    500,00 €
Keyboard  75,00 €
Paying 575,00 € using Bank Transfer
```

5 Adapter-Pattern erstellen

Erstelle eine Klasse `SelfPickupPaymentAdapter`, welche den `PickupService` bedient und das Interface `IPaymentStrategy` implementiert. Die Klasse `Person` hat eine Methode `GetCash()` um die Produkte zahlen zu können. Die neue Zahlungsmethode soll in der vorhanden Programmlogik ergänzt werden.

```
public class PickupService
{
    private decimal _balance;

    public Person Customer { get; }

    public PickupService(Person customer)
    {
        Customer = customer;
    }

    public void PayWithCash(decimal amount)
    {
        _balance = Customer.GetCash() - amount;
    }

    public decimal ReturnChange()
    {
        if (_balance < 0)
        {
            throw new InvalidOperationException("Not enough cash");
        }
        return _balance;
    }
}
```